

Mardik v2 — Analyse de contrats

Chantier 1 — La chaîne LLMOps · Chantier 2 — L'observabilité qui pilote

Livrables : synthèse du besoin · choix d'outils · contrat d'API v2 · chaîne LLMOps (branching, CI, CD) · métriques & boucles d'observabilité · tableau de pilotage · glossaires

Réf. brief.md / spec.md

CHANTIER 1

1 Synthèse du besoin

Un déploiement manuel raté a mis la production à l'arrêt une demi-journée (copie de fichiers à la main, aucune version identifiable, aucun retour arrière). Le CTO exige désormais une livraison 100 % automatisée, testée, observée et réversible.

FONCTIONNEL

- Documents longs : chunking contrôlé (découpage + agrégation), refus explicite **413** au-delà d'une limite absolue — jamais de troncature silencieuse
- confidence_score** (0–1) **global et par clause**
- /v1** strictement inchangée pour le client historique

CONTRAINTES

- Latence P95 < 8 s / analyse, y compris documents longs
- Coût moyen < 0,15 € / analyse (~400 contrats/mois)
- /v1** disponible en permanence, aucune interruption
- Zéro livraison manuelle — tout passe par la chaîne

HORS PÉRIMÈTRE (V2)

- Traduction automatique des contrats
- Chatbot conversationnel pour juristes
- Ré-entraînement du modèle
- UI avancée (au-delà d'un frontend minimal)

Point de vigilance non tranché — le chunking multiplie les appels LLM sur les documents longs (jusqu'à 40 pages) : risque de dépasser le P95 ou le coût cible sur ces cas. Mitigation retenue : appels parallélisés + nombre de chunks plafonné, surveillance par tranche de taille de document sur le tableau de bord plutôt qu'un dépassement silencieux.

Explication des choix d'outils non triviaux

Choix réellement ouverts du projet — hors outils déjà imposés par le sujet (Git, FastAPI, Pydantic) ou déjà fournis (télémetrie, golden dataset).

OUTIL RETENU	ALTERNATIVE ÉCARTÉE	RAISON DE L'ÉCART
 ORCHESTRATEUR CI/CD GitHub Actions Fait tourner <code>ci.yml</code> — jobs séquentiels <code>needs:</code>, co-localisé avec le dépôt et les règles de protection de <code>main</code>.	GitLab CI/CD	Le dépôt du projet n'est pas hébergé sur GitLab ; ajouter une plateforme CI séparée du dépôt introduit une synchronisation supplémentaire sans bénéfice
	Azure Pipelines	Pertinent au vu de l'écosystème .NET/Azure habituel, mais suppose un tenant Azure DevOps à provisionner pour un dépôt hébergé sur GitHub ; GitHub Actions reste natif au dépôt et suffit pour ce périmètre
	Jenkins	Nécessite d'héberger et de maintenir un serveur CI (agents, plugins, mises à jour) — overhead d'infrastructure disproportionné face à un pipeline déclaratif hébergé
 BACKEND DE TRACING Sentry Reçoit les traces OpenTelemetry (spans <code>analyse.requete / llm.appel</code>) — SDK OTel natif, managé, sans backend à héberger.	Jaeger	Auto-hébergé (stockage + UI à opérer) — infrastructure supplémentaire à maintenir alors que Sentry est managé et démarre sans serveur dédié
	Grafana Tempo	S'inscrit dans un écosystème à assembler (Tempo + Grafana + stockage objet) — overkill pour le volume de traces d'un dossier pédagogique
	Zipkin	Intégration OpenTelemetry moins directe (exporteur dédié à configurer) qu'un SDK Sentry nativement compatible OTel
 ROUTAGE CANARY Unleash Service de feature-flag gratuit/open-source — pilote le % de trafic v1/v2 par API (10 → 50 → 100 %), historisé.	Reverse proxy pondéré (Nginx / Envoy)	Routage au niveau infra uniquement — modifier un pourcentage exige de toucher la config du proxy et de la recharger, sans API de pilotage ni audit intégré
	Service mesh (Istio)	Suppose un cluster Kubernetes avec mesh installé — complexité d'infrastructure disproportionnée pour piloter un simple pourcentage de trafic
	Table de routage applicative maison	Réimplique de recoder la persistance des pourcentages, l'historique des changements et une interface de pilotage déjà fournis par un service de feature-flag dédié

Contrat d'API v2

Contrat figé avant développement (extrait OpenAPI 3.0, endpoint `/v2/analyse`) — enrichit `/v1` sans y toucher : `confidence_score` global et par clause, `warnings[]`, `model_version`, et un code `413` explicite en remplacement de la troncature silencieuse.

openapi.json — /v2/analyse

```
{
  "openapi": "3.0.3",
  "info": { "title": "Mardik Contract Analysis API", "version": "2.0.0" },
  "paths": {
    "/v2/analyse": {
      "post": {
        "summary": "Analyser un contrat – documents longs + score de confiance",
        "requestBody": {
          "required": true,
          "content": { "application/json": { "schema": {
            "type": "object",
            "required": ["texte"],
            "properties": {
              "texte": { "type": "string", "description": "Contenu du contrat à analyser" }
            }
          } }
        },
        "responses": {
          "200": {
            "description": "Analyse réalisée avec succès",
            "content": { "application/json": { "schema": {
              "type": "object",
              "properties": {
                "clauses": { "type": "array", // référentiel TYPES_CLAUSES, commun à v1/v2
                  "items": { "type": "object", "properties": {
                    "type": { "type": "string" },
                    "texte": { "type": "string" },
                    "confidence_score": { "type": "number", "minimum": 0, "maximum": 1 }
                  }
                },
                "confidence_score": {
                  "type": "number", "minimum": 0, "maximum": 1,
                  "description": "Score global du document"
                },
                "warnings": { "type": "array", "items": { "type": "string" } },
                "model_version": { "type": "string", "example": "mardik-v2.3.0-alb2c3d" },
                "modele": { "type": "string" },
                "version": { "type": "string" }
              }
            } }
          },
          "422": { "description": "Corps de requête invalide" },
          "413": {
            "description": "Document trop long – refus explicite, jamais de troncature silencieuse"
          },
          "503": { "description": "Fournisseur LLM indisponible" }
        }
      }
    }
  }
}
```

`/v1/analyse` garde son schéma actuel à l'identique (`clauses`, `modele`, `version`, `tronque`) — contrat figé, non représenté ici.

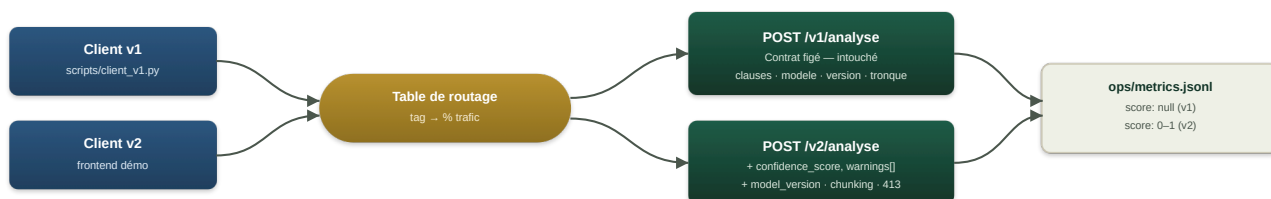
AGRÉGATION DES SCORES

- **Score document** = agrégation des scores de chunk (moyenne pondérée par longueur, ou minimum si prudence)
- **Score par clause** = score du chunk où elle est détectée, ou moyenne si répartie sur plusieurs chunks
- Méthode retenue documentée dans le changelog `model_version`, pour rester traçable et reproductible

VERSIONNAGE

- Versionnage de l'**API** par chemin d'URL : `/v1`, `/v2`, futur `/v3`
- Chaque version publiée a un contrat OpenAPI 3.0 **figé**, schémas de composants partagés entre versions
- `model_version` / `config_hash` versionnent le modèle et la config à l'intérieur d'une version d'API donnée
- Aucune dépréciation de `/v1` tant que le client historique en dépend

CLIENT V1 (INCHANGÉ) & CLIENT V2 — COEXISTENCE PENDANT LA TRANSITION



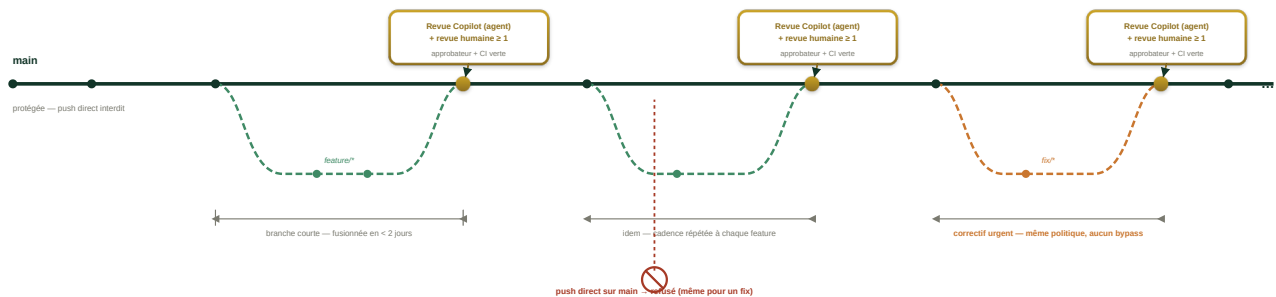
Coexistence `/v1` (contrat figé) et `/v2` (enrichie), routées par la même table de trafic canary, journalisées dans le même fichier de télémétrie.

Chaîne LLMOps

Trois schémas complémentaires : la politique de branching Git (4.1), le déclenchement des tests et gates en CI (4.2), puis la construction et le déploiement de l'artefact en CD (4.3).

4.1 Politique de branching Git

Trunk-based development : branches de feature courtes, toujours issues de **main** et fusionnées dans **main** — jamais de branche longue type **develop** / **release**. **main** est protégée : push direct interdit, fusion possible uniquement via PR avec double revue (agent GitHub Copilot + revue humaine ≥ 1 approbateur) + CI verte. Les correctifs (préfixe **fix/***) suivent exactement la **même politique** que les branches de feature — aucun bypass, même en urgence.

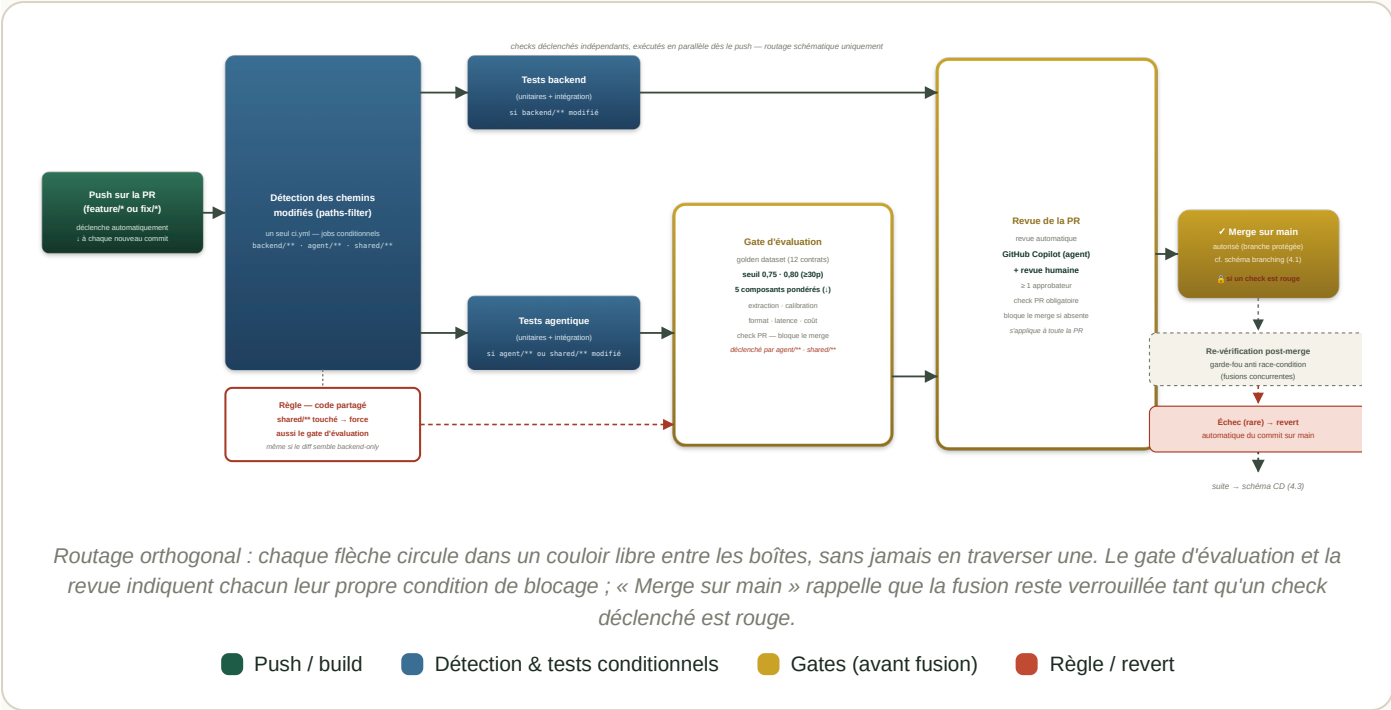


Chaque fonctionnalité (**feature/***) et chaque correctif (**fix/***) vivent dans une branche courte, fusionnée via PR — revue par un agent GitHub Copilot et par un humain (≥ 1 approbateur), CI verte obligatoire — jamais de push direct sur **main**, y compris en urgence.

● main (protégée) ● branche feature/* ● branche fix/* (même politique) ● fusion via PR (gate) ● push direct — refusé

4.2 Chaîne CI — déclenchement des tests et gates

Un seul `ci.yml`, décliné en jobs conditionnels par chemin modifié (*paths-filter*) : les tests backend et les tests/gate agentiques ne tournent que si leur périmètre est touché — mais tout changement sur du code **partagé** (`shared/**`) force systématiquement le gate d'évaluation, pour ne jamais valider le monolithe par morceaux. La revue de la PR (GitHub Copilot + humaine) couvre toujours l'intégralité de la PR, quel que soit le périmètre touché. **Tous les checks déclenchés tournent en parallèle dès le push** — le schéma les aligne pour la lisibilité, la fusion attend qu'ils soient *tous* verts.



Exemple — `ci.yml` (jobs conditionnels par chemin)

Un seul workflow GitHub Actions, avec un job de détection des chemins puis des jobs déclenchés conditionnellement — le job final `merge-gate` est le seul check requis par la protection de branche, et échoue si un job déclenché a échoué.

```
# Un seul pipeline – jobs conditionnels par chemin modifié
name: CI Mardik
on:
  pull_request:
    branches: [main]

jobs:
  changes:
    # détecte les chemins modifiés (schéma 4.2)
    runs-on: ubuntu-latest
    outputs:
      backend: ${ steps.filter.outputs.backend }
      agent: ${ steps.filter.outputs.agent }
    steps:
      - uses: actions/checkout@v4
      - uses: dorny/paths-filter@v3
        id: filter
        with:
          filters: |
            backend:
              - 'backend/**'
            agent:
              - 'agent/**'
              - 'shared/**' # code partagé -> force aussi l'agentique

  tests-backend:
    needs: changes
    if: needs.changes.outputs.backend == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest backend/tests/unit backend/tests/integration

  tests-agent:
    needs: changes
    if: needs.changes.outputs.agent == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest agent/tests

  gate-eval:
    # golden dataset – seuils 0,75 / 0,80 (§4.2)
    needs: [changes, tests-agent]
    if: needs.changes.outputs.agent == 'true'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          python ops/eval_gate.py --dataset attendus.jsonl \
            --seuil-base 0.75 --seuil-longs 0.80

  merge-gate:
    # seul check requis par la protection de branche (4.1)
    needs: [changes, tests-backend, tests-agent, gate-eval]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - run: |
          # échoue si un job déclenché (non "skipped") a échoué
          if [[ "${ steps.filter.outputs.backend }" == "true" ]]; then
            exit 1
          fi
```

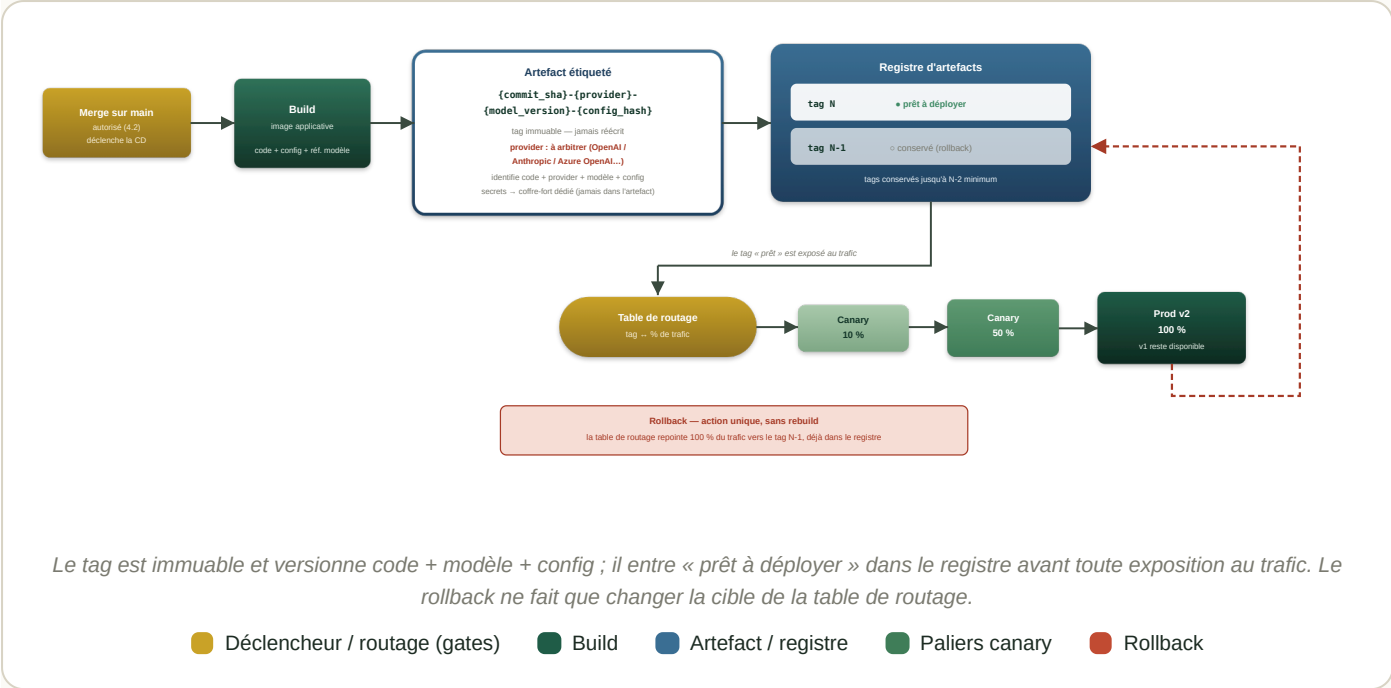
Zoom — composition du gate d'évaluation

Chaque composant est mesuré sur les 12 contrats du golden dataset. La note globale est une **moyenne pondérée** des composants, comparée au seuil (0,75 en base, 0,80 pour les contrats ≥ 30 pages) — pondération indicative, à calibrer avec les premiers retours de production.

COMPOSANT	POIDS INDICATIF	COMMENT IL EST MESURÉ
Extraction des clauses	35 %	Précision/rappel des clauses extraites vs <code>clauses_attendues</code> du golden dataset (par contrat, puis moyenné)
Calibration du score de confiance	25 %	Écart entre <code>confidence_score</code> annoncé et exactitude réelle observée (ex. Brier score / ECE) — un score de 0,9 doit correspondre à ~90 % de bonnes extractions
Respect du format de réponse	15 %	Validation stricte du schéma JSON de sortie (Pydantic) — pass/fail par contrat, sans champ manquant ni type invalide
Latence sur le golden dataset	15 %	P95 des 12 contrats vs la contrainte < 8 s, y compris les contrats longs multi-chunks
Coût sur le golden dataset	10 %	Coût moyen par contrat vs la contrainte < 0,15 €/analyse

4.3 Chaîne CD — artefact étiqueté & déploiement

Une fois le merge autorisé (4.2), l'image est construite, étiquetée de façon **immuable** ({commit_sha}-{provider}-{model_version}-{config_hash}) — le provider LLM n'est pas encore arbitré et placée dans le registre d'artefacts en état « **prêt à déployer** » — avant même que la table de routage ne l'expose au trafic. Le rollback ne fait que repointer le trafic vers un tag déjà présent dans le registre : aucun rebuild.

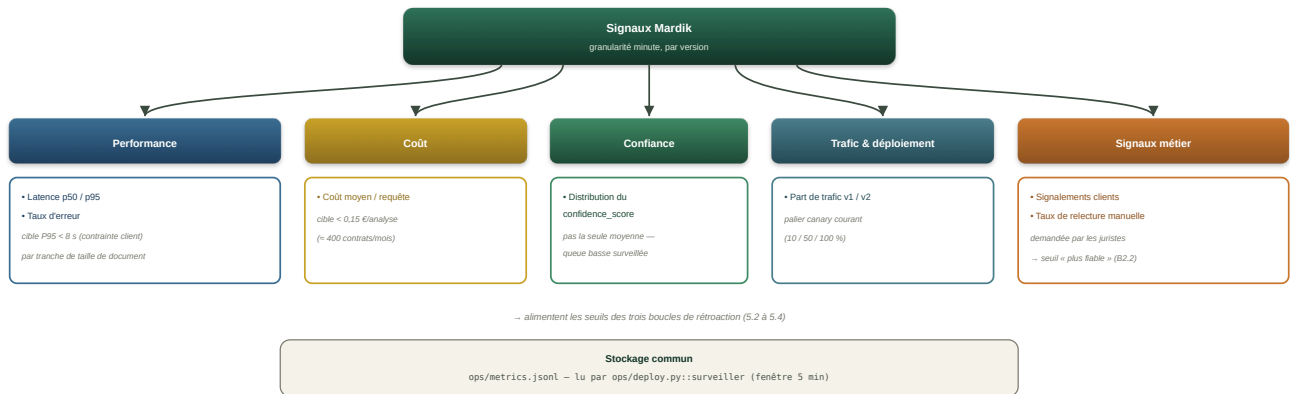


Boucle d'observabilité

Cinq schémas complémentaires : les métriques et leurs signaux associés (5.1), chacune des trois boucles de rétroaction prise isolément (5.2 rollback, 5.3 promotion canary, 5.4 enrichissement du golden dataset), puis un exemple concret de trace avec ses spans par composant (5.5).

5.1 Métriques et signaux associés

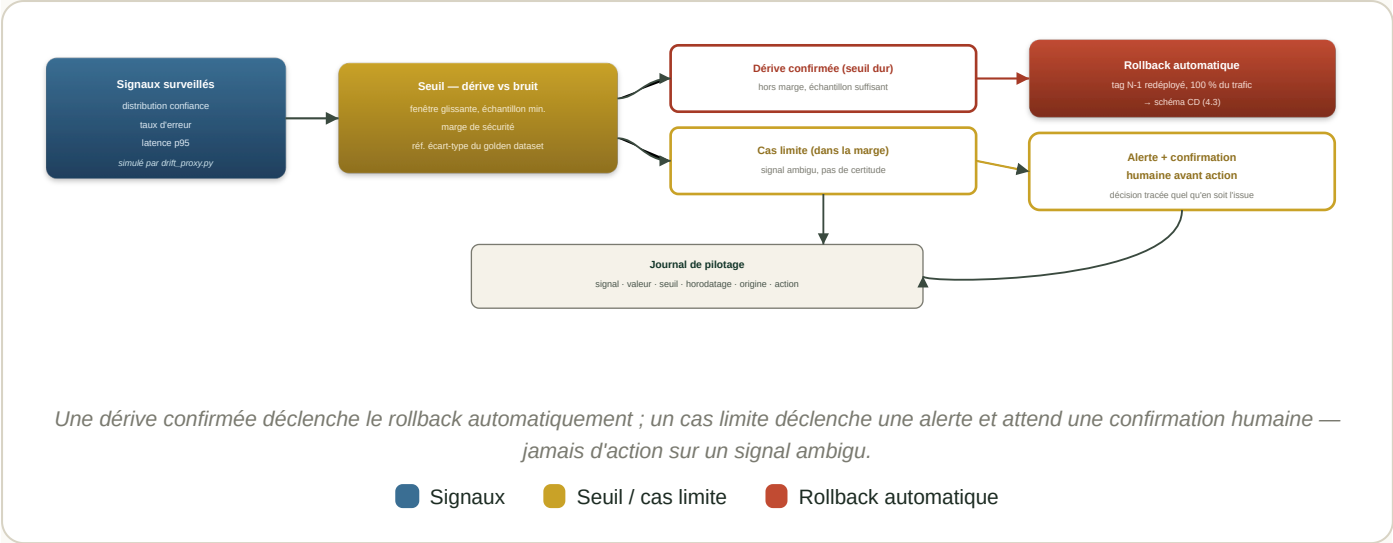
La v2 réutilise telle quelle l'infrastructure de télémétrie existante (`app/telemetry.py` , spans, `MetricsStore` → `ops/metrics.jsonl`), en renseignant le champ `score` (`null` en v1) — instrumentation à granularité minute, par version.



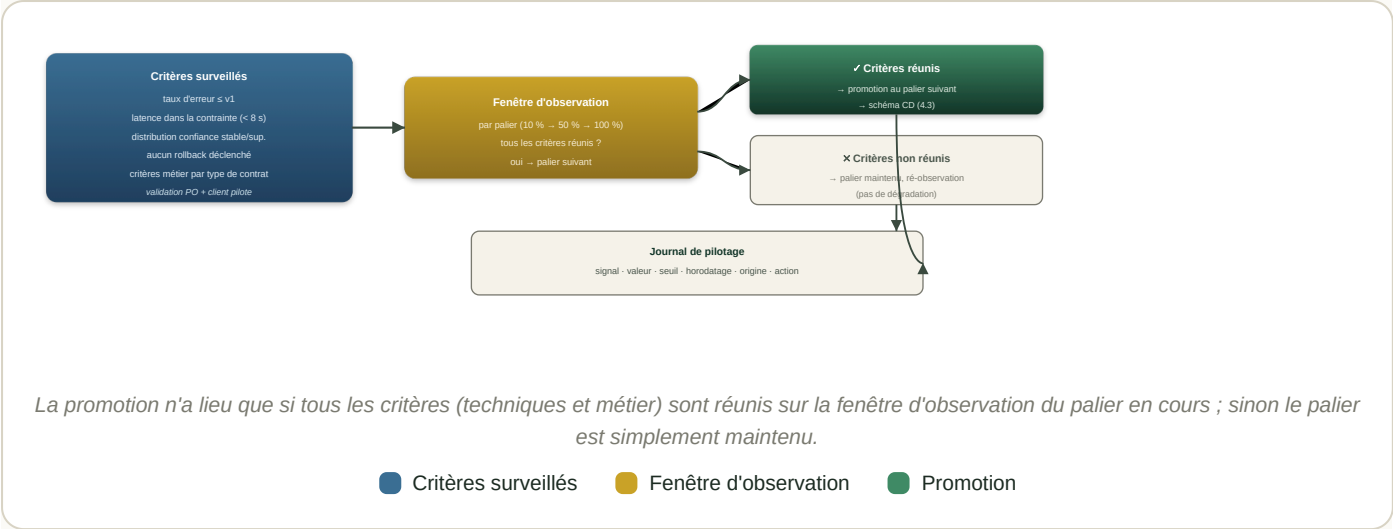
Cinq catégories de signaux, toutes instrumentées à la minute et par version — socle commun des trois boucles de rétroaction ci-après.

■ Performance ■ Coût ■ Confiance ■ Trafic & déploiement ■ Signaux métier

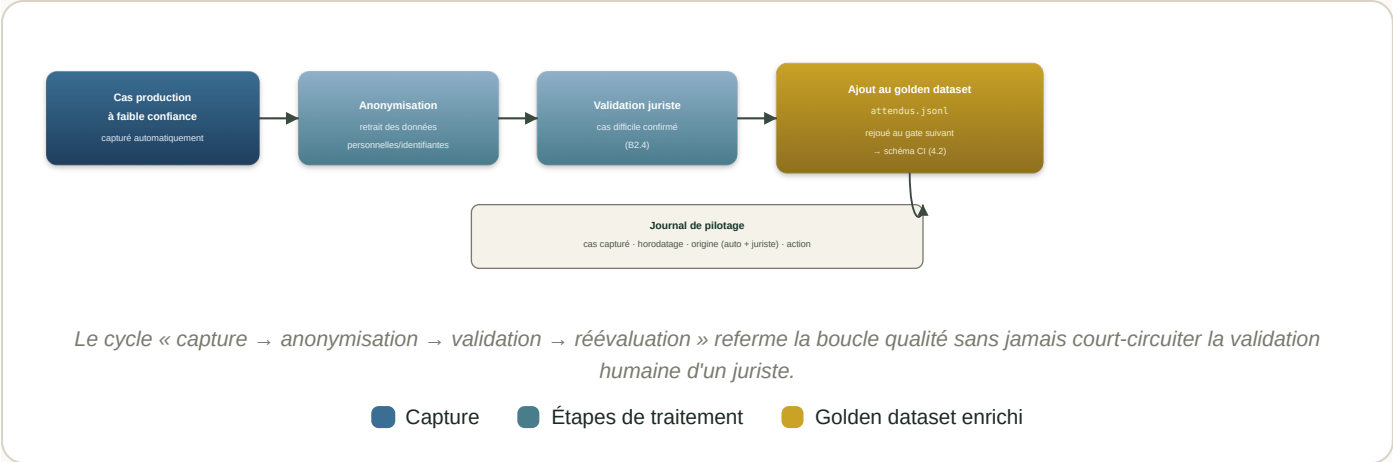
5.2 Boucle de rétroaction — rollback



5.3 Boucle de rétroaction — promotion canary

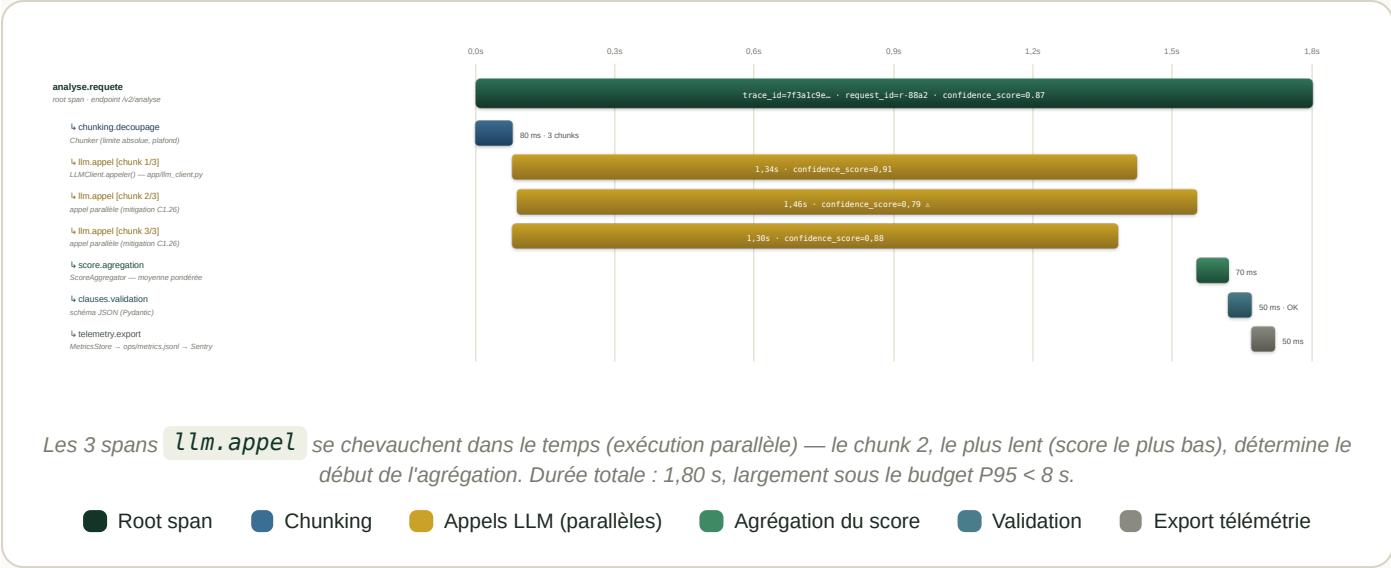


5.4 Boucle de rétroaction — enrichissement du golden dataset



5.5 Exemple de trace — spans par composant

Une trace = l'arbre complet des spans d'une requête, identifiée par un `trace_id` unique et propagée de bout en bout (C2.13). Exemple ci-dessous : une requête `/v2/analyse` sur un contrat découpé en 3 chunks, appels LLM parallélisés (mitigation C1.26).



SPAN	COMPOSANT MARDIK	ATTRIBUTS CLÉS
<code>analyse.requete</code>	Endpoint FastAPI <code>/v2/analyse</code>	<code>trace_id</code> , <code>request_id</code> , <code>model_version</code> , <code>confidence_score</code> (global)
<code>chunking.decoupage</code>	Chunker (découpage + limite absolue)	nombre de chunks, taille du document
<code>llm.appeal</code> (×3, parallèles)	LLMClient / Bundle — <code>app/llm_client.py</code>	index du chunk, tokens in/out, coût, <code>confidence_score</code> par chunk
<code>score.agregation</code>	Agrégateur de score (méthode C1.23)	méthode (moyenne pondérée / minimum)
<code>clauses.validation</code>	Validation de schéma (Pydantic)	statut pass/fail, champ en erreur le cas échéant
<code>telemetry.export</code>	<code>MetricsStore</code> → <code>ops/metrics.jsonl</code> → Sentry	span exporté, ligne <code>Mesure</code> écrite à la minute

`trace_id` et `request_id` sont propagés sur tous les spans de la requête — ce qui permet, par exemple, de retrouver la trace d'origine d'un cas versé au golden dataset (5.4) à partir de son `request_id` journalisé.

Tableau de pilotage

Vue synthétique : signal surveillé × seuil × rétroaction déclenchée × contenu tracé au journal de pilotage.

SIGNAL	SEUIL	RÉTROACTION DÉCLENCHÉE	TRACE AU JOURNAL
Score de confiance moyen glissant	Sous seuil dur, N requêtes consécutives (fenêtre glissante, échantillon min.)	Rollback automatique — rejoue le tag N-1 via la chaîne	signal, valeur mesurée, seuil, horodatage, origine= <i>auto</i> , action=rollback
Score de confiance — cas limite	Dans la marge de sécurité, sous le seuil dur	Alerte + confirmation humaine avant action	signal, horodatage, origine= <i>humaine (après alerte)</i> , action=à décider
Taux d'erreur canary	≤ taux d'erreur v1	Poursuite promotion — 10 % → 50 % → 100 % si <i>tous</i> les critères sont tenus sur la fenêtre d'observation	signal, valeurs mesurées, seuils, horodatage, origine= <i>auto</i> (+ validation PO/client pilote), action=promotion
Latence P95 canary	< 8 secondes		
Distribution de confiance canary	Stable ou supérieure à la référence golden dataset		
Critère métier par type de contrat	Conforme (types de contrats prioritaires)		
Cas de production à faible confiance	Score sous le seuil	Capture → anonymisation → validation juriste → ajout au golden dataset	signal, cas capturé, horodatage, origine= <i>auto (capture)</i> + <i>humaine (validation)</i> , action=enrichissement
Seuil de gate lui-même	Ajustement décidé (retour d'expérience prod.)	Commit versionné (config as code) — jamais en silence	ancien seuil, nouveau seuil, message expliquant la raison, auteur, horodatage

Langage ubiquitaire (extrait)

Indice de fiabilité / niveau de certitude

vocabulaire métier du `confidence_score` technique — probabilité que l'analyse soit correcte, aux yeux du juriste.

Publiable

version au statut CI vert, prête à livrer.

Version fiable

golden dataset intégrant des critères qualité métier, pas seulement une note technique.

Incident

contrat mal analysé livré avec impact client ; notifié au CTO, au client pilote et au support.

Cas difficile

contrat aux clauses ambiguës ou atypiques, validé par un juriste avant intégration au golden dataset.

Audit non-technicien

journal de pilotage lisible en langage métier (ex. « version retirée car X % des analyses jugées peu fiables »).

Glossaires par chantier

Vocabulaire technique utilisé dans les schémas et tableaux de chaque chantier, regroupé pour référence rapide.

7.1 Chantier 1 — Chaîne LLMOps

Trunk-based development

stratégie où les branches de feature sont courtes et fusionnées fréquemment dans une unique branche principale (`main`), sans branches longues type `develop` / `release` .

Branche protégée

branche (`main`) sur laquelle le push direct est interdit ; toute fusion exige une revue et un statut CI vert.

Pull Request (PR)

proposition de fusion de code soumise à revue humaine et à des vérifications automatiques avant intégration.

Gate bloquant

étape de vérification automatique (test, évaluation) qui arrête le pipeline si elle échoue — aucune étape suivante ne s'exécute.

Golden dataset

jeu de contrats de référence annotés (`attendus.jsonl`), utilisé pour évaluer automatiquement la qualité d'une version du modèle.

Artefact étiqueté

image versionnée de façon immuable par un tag combinant commit, provider LLM, version de modèle et hash de configuration (`{commit_sha}-{provider}-{model_version}-{config_hash}`).

model_version / config_hash

identifiants versionnant respectivement le modèle et la configuration utilisés à l'intérieur d'une version d'API donnée.

Registre d'artefacts

emplacement où les artefacts construits sont stockés en état « prêt à déployer », avant toute exposition au trafic.

Déploiement canary

stratégie exposant une nouvelle version à une fraction croissante du trafic (10 → 50 → 100 %) avant généralisation.

Table de routage

composant associant chaque tag d'artefact à un pourcentage de trafic, à un instant donné.

Rollback

retour à la version précédente (tag N-1) sans reconstruction — la table de routage est simplement repointée.

OpenAPI

spécification standard décrivant le contrat d'une API REST (endpoints, schémas de données, codes d'erreur).

Chunking

découpage d'un document long en segments traités séparément, puis agrégation des résultats (score, clauses).

Feature-flag

mécanisme logiciel pilotant l'activation ou le dosage d'une fonctionnalité (ici, le pourcentage de trafic) sans redéploiement.

7.2 Chantier 2 — Observabilité

Span

unité de trace représentant une opération (ex. un appel LLM) dans un système de tracing distribué (OpenTelemetry).

OpenTelemetry (OTel)

standard ouvert d'instrumentation pour collecter traces, métriques et logs de façon uniforme.

P50 / P95

valeurs de latence sous lesquelles se situent respectivement 50 % et 95 % des requêtes — indicateurs de performance typique et de pire cas.

Fenêtre glissante

intervalle de temps ou nombre de requêtes récentes utilisé pour calculer une métrique en continu (ex. 5 minutes, 500 requêtes).

Dérive (drift)

écart progressif entre le comportement observé en production et le comportement de référence (ex. baisse du score de confiance).

MetricsStore / ops/metrics.jsonl

composant et fichier stockant les mesures structurées (latence, coût, erreur, score) par requête et par version.

Journal de pilotage

registre traçant chaque ajustement (seuil, promotion, rollback) : signal déclencheur, valeur mesurée, seuil, horodatage, origine.

Garde-fou anti race-condition

vérification de sécurité exécutée après un événement (ici, après fusion) pour couvrir un cas rare non détecté avant coup.

Indice de fiabilité / niveau de certitude

terme métier du `confidence_score` technique, ce qu'il signifie concrètement pour le juriste.